```
-- Template | GUI.
-- Services.
local playersService = game:GetService("Players")
local userInputService = game:GetService("UserInputService")
local tweenService = game:GetService("TweenService")
local httpService = game:GetService("HttpService")

-- Constants.
local WINDOW_WIDTH = 420
local WINDOW_HEIGHT = 540
local TITLE_HEIGHT = 34
local TAB_HEIGHT = 30
local MIN_WIDTH = 360
local MIN_HEIGHT = 360
local MAX_WIDTH = 820
local MAX_HEIGHT = 800
local CONFIG_FOLDER = "Template"
local CONFIG_FILE = "Template/Template.json"
local TAB_NAMES = { "Tab1", "Tab2", "Tab3", "Config" }
local TITLE_TEXT = "Template"

local ACCENT_A = Color3.fromRGB(225, 225, 225)
local ACCENT_B = Color3.fromRGB(110, 110, 116)
local BACKGROUND_TOP = Color3.fromRGB(25, 25, 27)
local BACKGROUND_BOTTOM = Color3.fromRGB(9, 9, 10)
local ROW_COLOR = Color3.fromRGB(34, 34, 36)
local ELEMENT_COLOR = Color3.fromRGB(17, 17, 19)
local TEXT_COLOR = Color3.fromRGB(238, 238, 238)
local OFF_TEXT_COLOR = Color3.fromRGB(192, 192, 196)
local DIM_TEXT_COLOR = Color3.fromRGB(120, 120, 124)

local TEXT_FONT = Enum.Font.SourceSans
local BOLD_FONT = Enum.Font.SourceSansBold

local FAST_TWEEN = TweenInfo.new(0.15, Enum.EasingStyle.Quad, Enum.EasingDirection.Out)
local SLIDE_TWEEN = TweenInfo.new(0.08, Enum.EasingStyle.Quad, Enum.EasingDirection.Out)
local MEDIUM_TWEEN = TweenInfo.new(0.35, Enum.EasingStyle.Quint, Enum.EasingDirection.Out)
local POP_TWEEN = TweenInfo.new(0.3, Enum.EasingStyle.Back, Enum.EasingDirection.Out)

local localPlayer = playersService.LocalPlayer

local TemplateHub = {
    values = {},
    pages = {},
    tabButtons = {},
    tabIndex = {},
    searchables = {},
    sections = {},
    boxes = {},
    sectionBoxes = {},
    setters = {},
    connections = {},
    keybinds = {},
    keyLabels = {},
    listening = nil,
    defaults = {},
    menuKey = Enum.KeyCode.RightShift,
}

---Create an instance with properties.
local function create(className, properties, parent)
    local instance = Instance.new(className)

    for key, value in properties do
        instance[key] = value
    end

    instance.Parent = parent
    return instance
end

---Play a tween.
local function tween(instance, info, goal)
    local newTween = tweenService:Create(instance, info, goal)
    newTween:Play()
    return newTween
end

---Connect a global input signal that is cleaned up when the GUI closes.
local function connectInput(signal, callback)
    local connection = signal:Connect(callback)
    table.insert(TemplateHub.connections, connection)
    return connection
```

```
end

---Check if an input is a primary press.
local function isPress(input)
    return input.UserInputType == Enum.UserInputType.MouseButton1 or input.UserInputType == Enum.UserInputType.Touch
end

---Check if an input is pointer movement.
local function isMove(input)
    return input.UserInputType == Enum.UserInputType.MouseMovement or input.UserInputType == Enum.UserInputType.Touch
end

---Add the accent gradient to an object.
local function addGradient(parent, rotation)
    return create("UIGradient", {
        Color = ColorSequence.new(ACCENT_A, ACCENT_B),
        Rotation = rotation,
    }, parent)
end

---Add a border stroke to an object.
local function addStroke(parent, color, transparency, thickness)
    return create("UIStroke", {
        Color = color,
        Transparency = transparency,
        Thickness = thickness or 1,
        ApplyStrokeMode = Enum.ApplyStrokeMode.Border,
    }, parent)
end

---Create a rounded row card.
local function createCard(className, parent, text, height)
    local card = create(className, {
        Name = text,
        Size = UDim2.new(1, 0, 0, height),
        BackgroundColor3 = ROW_COLOR,
        BackgroundTransparency = 0.65,
        BorderSizePixel = 0,
    }, TemplateHub.sectionBoxes[parent] or parent)

    create("UICorner", { CornerRadius = UDim.new(0, 5) }, card)
    local stroke = addStroke(card, ACCENT_A, 1)
    card.ClipsDescendants = true

    -- Rows of a collapsible group start hidden and follow the group button.
    local group = TemplateHub.currentGroup
    if group then
        table.insert(group.cards, card)
        card:SetAttribute("GroupHidden", group.collapsed)
        card.Visible = not group.collapsed
    end

    -- Click ripple that expands from the cursor.
    card.InputBegan:Connect(function(input)
        if not isPress(input) then
            return
        end

        local offset = Vector2.new(input.Position.X, input.Position.Y) - card.AbsolutePosition

        local ripple = create("Frame", {
            AnchorPoint = Vector2.new(0.5, 0.5),
            Position = UDim2.fromOffset(offset.X, offset.Y),
            Size = UDim2.fromOffset(0, 0),
            BackgroundColor3 = Color3.new(1, 1, 1),
            BackgroundTransparency = 0.8,
            BorderSizePixel = 0,
            ZIndex = 0,
        }, card)

        create("UICorner", { CornerRadius = UDim.new(1, 0) }, ripple)

        local diameter = math.max(card.AbsoluteSize.X, card.AbsoluteSize.Y) * 2

        tween(ripple, MEDIUM_TWEEN, {
            Size = UDim2.fromOffset(diameter, diameter),
            BackgroundTransparency = 1,
        }).Completed:Connect(function()
            ripple:Destroy()
        end)
    end)

    card.MouseEnter:Connect(function()
```

```
        tween(card, FAST_TWEEN, { BackgroundTransparency = 0.35 })
        tween(stroke, FAST_TWEEN, { Transparency = 0.55 })
    end)

    card.MouseLeave:Connect(function()
        tween(card, FAST_TWEEN, { BackgroundTransparency = 0.65 })
        tween(stroke, FAST_TWEEN, { Transparency = 1 })
    end)

    return card, stroke
end

---Add a section group box. Following rows are placed inside it.
local function addSection(parent, text)
    local box = create("Frame", {
        Name = text .. " Box",
        Size = UDim2.new(1, 0, 0, 0),
        AutomaticSize = Enum.AutomaticSize.Y,
        BackgroundColor3 = Color3.fromRGB(22, 22, 24),
        BackgroundTransparency = 0.2,
        BorderSizePixel = 0,
        LayoutOrder = #TemplateHub.boxes + 1,
    }, parent)

    create("UICorner", { CornerRadius = UDim.new(0, 8) }, box)
    addStroke(box, Color3.fromRGB(105, 105, 110), 0.55)

    create("UIGradient", {
        Color = ColorSequence.new(Color3.new(1, 1, 1), Color3.fromRGB(165, 165, 170)),
        Rotation = 90,
    }, box)

    create("UIPadding", {
        PaddingTop = UDim.new(0, 4),
        PaddingBottom = UDim.new(0, 6),
        PaddingLeft = UDim.new(0, 6),
        PaddingRight = UDim.new(0, 6),
    }, box)

    create("UIListLayout", { Padding = UDim.new(0, 3), SortOrder = Enum.SortOrder.LayoutOrder }, box)
    local holder = create("Frame", {
        Name = text,
        Size = UDim2.new(1, 0, 0, 26),
        BackgroundTransparency = 1,
    }, box)

    local marker = create("Frame", {
        Size = UDim2.fromOffset(3, 12),
        Position = UDim2.new(0, 3, 0.5, -7),
        BackgroundColor3 = Color3.new(1, 1, 1),
        BorderSizePixel = 0,
    }, holder)

    create("UICorner", { CornerRadius = UDim.new(1, 0) }, marker)
    addGradient(marker, 90)

    tweenService:Create(
        marker,
        TweenInfo.new(1.6, Enum.EasingStyle.Sine, Enum.EasingDirection.InOut, -1, true),
        { BackgroundTransparency = 0.5 }
    ):Play()

    create("TextLabel", {
        Size = UDim2.new(1, -14, 1, -4),
        Position = UDim2.fromOffset(12, 0),
        BackgroundTransparency = 1,
        Text = string.upper(text),
        TextColor3 = ACCENT_A,
        TextSize = 11,
        Font = BOLD_FONT,
        TextXAlignment = Enum.TextXAlignment.Left,
    }, holder)

    local line = create("Frame", {
        Size = UDim2.new(1, 0, 0, 1),
        Position = UDim2.new(0, 0, 1, -2),
        BackgroundColor3 = Color3.new(1, 1, 1),
        BorderSizePixel = 0,
    }, holder)

    local gradient = addGradient(line, 0)
    gradient.Transparency = NumberSequence.new(0.3, 1)
```

```
    TemplateHub.sectionBoxes[parent] = box
    table.insert(TemplateHub.boxes, box)
    table.insert(TemplateHub.sections, holder)
end

---Add a toggle row.
local function addToggle(parent, text, keybind, default)
    local holder, stroke = createCard("TextButton", parent, text, 28)
    holder.Text = ""
    holder.AutoButtonColor = false

    local box = create("Frame", {
        Size = UDim2.fromOffset(16, 16),
        Position = UDim2.new(0, 8, 0.5, -8),
        BackgroundColor3 = ELEMENT_COLOR,
        BorderSizePixel = 0,
    }, holder)

    create("UICorner", { CornerRadius = UDim.new(0, 4) }, box)
    local boxStroke = addStroke(box, default and ACCENT_A or DIM_TEXT_COLOR, default and 0 or 0.5)

    local check = create("Frame", {
        AnchorPoint = Vector2.new(0.5, 0.5),
        Position = UDim2.fromScale(0.5, 0.5),
        Size = default and UDim2.fromOffset(10, 10) or UDim2.fromOffset(0, 0),
        BackgroundColor3 = Color3.new(1, 1, 1),
        BorderSizePixel = 0,
    }, box)

    create("UICorner", { CornerRadius = UDim.new(0, 3) }, check)
    addGradient(check, 45)

    local label = create("TextLabel", {
        Size = UDim2.new(1, -40, 1, 0),
        Position = UDim2.fromOffset(32, 0),
        BackgroundTransparency = 1,
        Text = text,
        TextColor3 = default and TEXT_COLOR or OFF_TEXT_COLOR,
        TextSize = 13,
        Font = TEXT_FONT,
        TextXAlignment = Enum.TextXAlignment.Left,
    }, holder)

    -- Glowing marker on the left edge while enabled.
    local accentBar = create("Frame", {
        Size = UDim2.new(0, 3, 1, -10),
        Position = UDim2.fromOffset(0, 5),
        BackgroundColor3 = Color3.new(1, 1, 1),
        BackgroundTransparency = default and 0 or 1,
        BorderSizePixel = 0,
    }, holder)

    create("UICorner", { CornerRadius = UDim.new(1, 0) }, accentBar)
    addGradient(accentBar, 90)

    local state = default
    TemplateHub.values[text] = state
    TemplateHub.defaults[text] = default

    TemplateHub.setters[text] = function(value)
        state = value == true
        TemplateHub.values[text] = state

        tween(check, POP_TWEEN, { Size = state and UDim2.fromOffset(10, 10) or UDim2.fromOffset(0, 0) })
        tween(boxStroke, FAST_TWEEN, { Color = state and ACCENT_A or DIM_TEXT_COLOR, Transparency = state and 0 or 0.5 })
        tween(label, FAST_TWEEN, { TextColor3 = state and TEXT_COLOR or OFF_TEXT_COLOR })
        tween(accentBar, FAST_TWEEN, { BackgroundTransparency = state and 0 or 1 })
    end

    -- Clickable keybind label: click it, then press a key. Escape clears it.
    if keybind then
        local keybindName = text .. " Keybind"

        local keybindButton = create("TextButton", {
            Size = UDim2.fromOffset(52, 22),
            Position = UDim2.new(1, -58, 0.5, -11),
            BackgroundTransparency = 1,
            Text = "[" .. keybind .. "]",
            TextColor3 = DIM_TEXT_COLOR,
            TextSize = 12,
            Font = TEXT_FONT,
            TextXAlignment = Enum.TextXAlignment.Right,
```

```
            AutoButtonColor = false,
        }, holder)

        ---Apply a key name to this toggle.
        local function applyKey(keyName)
            local keyCode = nil

            for _, item in Enum.KeyCode:GetEnumItems() do
                if item.Name == keyName then
                    keyCode = item
                    break
                end
            end

            TemplateHub.keybinds[text] = keyCode
            TemplateHub.values[keybindName] = keyCode and keyName or ""
            keybindButton.Text = keyCode and ("[" .. keyName .. "]") or "[None]"
            keybindButton.Visible = TemplateHub.values["Keybind Hints"] ~= false
        end

        TemplateHub.defaults[keybindName] = keybind
        TemplateHub.setters[keybindName] = applyKey
        TemplateHub.keyLabels[text] = keybindButton
        applyKey(keybind)

        keybindButton.MouseEnter:Connect(function()
            tween(keybindButton, FAST_TWEEN, { TextColor3 = ACCENT_A })
        end)

        keybindButton.MouseLeave:Connect(function()
            if not TemplateHub.listening then
                tween(keybindButton, FAST_TWEEN, { TextColor3 = DIM_TEXT_COLOR })
            end
        end)

        keybindButton.Activated:Connect(function()
            -- Cancel any other keybind that is still waiting for a key.
            if TemplateHub.cancelListening then
                TemplateHub.cancelListening()
            end

            local previousKey = TemplateHub.values[keybindName]
            keybindButton.Text = "[...]"
            tween(keybindButton, FAST_TWEEN, { TextColor3 = ACCENT_A })

            TemplateHub.cancelListening = function()
                TemplateHub.listening = nil
                TemplateHub.cancelListening = nil
                tween(keybindButton, FAST_TWEEN, { TextColor3 = DIM_TEXT_COLOR })
                applyKey(previousKey)
            end

            TemplateHub.listening = function(keyCode)
                TemplateHub.listening = nil
                TemplateHub.cancelListening = nil
                tween(keybindButton, FAST_TWEEN, { TextColor3 = DIM_TEXT_COLOR })
                applyKey(keyCode == Enum.KeyCode.Escape and "" or keyCode.Name)
            end
        end)
    end

    holder.MouseButton1Down:Connect(function()
        tween(holder, FAST_TWEEN, { BackgroundTransparency = 0 })
        tween(stroke, FAST_TWEEN, { Transparency = 0 })
    end)

    holder.MouseButton1Up:Connect(function()
        tween(holder, FAST_TWEEN, { BackgroundTransparency = 0.35 })
        tween(stroke, FAST_TWEEN, { Transparency = 0.55 })
    end)

    holder.Activated:Connect(function()
        TemplateHub.setters[text](not state)

        if TemplateHub.notify then
            TemplateHub.notify(text, state)
        end
    end)

    holder.MouseEnter:Connect(function()
        tween(label, FAST_TWEEN, { Position = UDim2.fromOffset(36, 0) })
    end)
```

```
    holder.MouseLeave:Connect(function()
        tween(label, FAST_TWEEN, { Position = UDim2.fromOffset(32, 0) })
    end)

    table.insert(TemplateHub.searchables, holder)
end

---Add a slider row.
local function addSlider(parent, text, minValue, maxValue, default, decimals)
    local holder = createCard("Frame", parent, text, 44)
    local startAlpha = (default - minValue) / (maxValue - minValue)

    create("TextLabel", {
        Size = UDim2.new(1, -70, 0, 20),
        Position = UDim2.fromOffset(10, 3),
        BackgroundTransparency = 1,
        Text = text,
        TextColor3 = TEXT_COLOR,
        TextSize = 13,
        Font = TEXT_FONT,
        TextXAlignment = Enum.TextXAlignment.Left,
    }, holder)

    local valueLabel = create("TextLabel", {
        Size = UDim2.fromOffset(56, 20),
        Position = UDim2.new(1, -66, 0, 3),
        BackgroundTransparency = 1,
        Text = tostring(default),
        TextColor3 = OFF_TEXT_COLOR,
        TextSize = 13,
        Font = BOLD_FONT,
        TextXAlignment = Enum.TextXAlignment.Right,
    }, holder)

    local bar = create("Frame", {
        Size = UDim2.new(1, -20, 0, 6),
        Position = UDim2.fromOffset(10, 29),
        BackgroundColor3 = ELEMENT_COLOR,
        BorderSizePixel = 0,
    }, holder)
    create("UICorner", { CornerRadius = UDim.new(1, 0) }, bar)
    addStroke(bar, Color3.new(1, 1, 1), 0.88)

    local fill = create("Frame", {
        Size = UDim2.fromScale(startAlpha, 1),
        BackgroundColor3 = Color3.new(1, 1, 1),
        BorderSizePixel = 0,
    }, bar)

    create("UICorner", { CornerRadius = UDim.new(1, 0) }, fill)
    addGradient(fill, 0)

    local knob = create("Frame", {
        AnchorPoint = Vector2.new(0.5, 0.5),
        Size = UDim2.fromOffset(12, 12),
        Position = UDim2.new(startAlpha, 0, 0.5, 0),
        BackgroundColor3 = Color3.new(1, 1, 1),
        BorderSizePixel = 0,
    }, bar)

    create("UICorner", { CornerRadius = UDim.new(1, 0) }, knob)
    addStroke(knob, ACCENT_A, 0, 2)
    local knobScale = create("UIScale", {}, knob)

    local hitbox = create("Frame", {
        Size = UDim2.new(1, -20, 0, 22),
        Position = UDim2.fromOffset(10, 21),
        BackgroundTransparency = 1,
    }, holder)

    TemplateHub.values[text] = default
    TemplateHub.defaults[text] = default

    local dragging = false

    local function setValue(value)
        local factor = 10 ^ decimals
        value = math.floor(math.clamp(value, minValue, maxValue) * factor + 0.5) / factor
        local snapped = (value - minValue) / (maxValue - minValue)

        tween(fill, SLIDE_TWEEN, { Size = UDim2.fromScale(snapped, 1) })
        tween(knob, SLIDE_TWEEN, { Position = UDim2.new(snapped, 0, 0.5, 0) })
```

```
        valueLabel.Text = tostring(value)
        TemplateHub.values[text] = value
    end

    TemplateHub.setters[text] = setValue

    local function updateFromX(positionX)
        local alpha = math.clamp((positionX - bar.AbsolutePosition.X) / bar.AbsoluteSize.X, 0, 1)
        setValue(minValue + (maxValue - minValue) * alpha)
    end

    hitbox.InputBegan:Connect(function(input)
        if not isPress(input) then
            return
        end

        dragging = true
        tween(knobScale, POP_TWEEN, { Scale = 1.35 })
        tween(valueLabel, FAST_TWEEN, { TextColor3 = ACCENT_A })
        updateFromX(input.Position.X)
    end)

    connectInput(userInputService.InputChanged, function(input)
        if dragging and isMove(input) then
            updateFromX(input.Position.X)
        end
    end)

    connectInput(userInputService.InputEnded, function(input)
        if not dragging or not isPress(input) then
            return
        end

        dragging = false
        tween(knobScale, POP_TWEEN, { Scale = 1 })
        tween(valueLabel, FAST_TWEEN, { TextColor3 = OFF_TEXT_COLOR })
    end)

    table.insert(TemplateHub.searchables, holder)
end

---Add a cycling option row.
local function addCycle(parent, text, options)
    local holder = createCard("Frame", parent, text, 56)

    create("TextLabel", {
        Size = UDim2.new(1, -20, 0, 20),
        Position = UDim2.fromOffset(10, 3),
        BackgroundTransparency = 1,
        Text = text,
        TextColor3 = TEXT_COLOR,
        TextSize = 13,
        Font = TEXT_FONT,
        TextXAlignment = Enum.TextXAlignment.Left,
    }, holder)

    local button = create("TextButton", {
        Size = UDim2.new(1, -20, 0, 24),
        Position = UDim2.fromOffset(10, 26),
        BackgroundColor3 = ELEMENT_COLOR,
        BorderSizePixel = 0,
        Text = options[1],
        TextColor3 = TEXT_COLOR,
        TextSize = 13,
        Font = TEXT_FONT,
        AutoButtonColor = false,
    }, holder)

    create("UICorner", { CornerRadius = UDim.new(0, 4) }, button)
    local buttonStroke = addStroke(button, DIM_TEXT_COLOR, 0.6)

    local index = 1
    TemplateHub.values[text] = options[1]
    TemplateHub.defaults[text] = options[1]

    TemplateHub.setters[text] = function(value)
        local found = table.find(options, value)
        if not found then
            return
        end

        index = found
```

```
        TemplateHub.values[text] = options[index]
        button.Text = options[index]
    end

    button.MouseEnter:Connect(function()
        tween(buttonStroke, FAST_TWEEN, { Color = ACCENT_A, Transparency = 0 })
    end)

    button.MouseLeave:Connect(function()
        tween(buttonStroke, FAST_TWEEN, { Color = DIM_TEXT_COLOR, Transparency = 0.6 })
    end)

    button.Activated:Connect(function()
        index = index % #options + 1
        TemplateHub.values[text] = options[index]

        button.Text = options[index]
        button.TextTransparency = 1
        tween(button, FAST_TWEEN, { TextTransparency = 0 })
    end)

    table.insert(TemplateHub.searchables, holder)
end

---Add an action button row.
local function addButton(parent, text, callback)
    local holder = createCard("Frame", parent, text, 34)

    local button = create("TextButton", {
        Size = UDim2.new(1, -16, 0, 24),
        Position = UDim2.fromOffset(8, 5),
        BackgroundColor3 = ELEMENT_COLOR,
        BorderSizePixel = 0,
        Text = text,
        TextColor3 = TEXT_COLOR,
        TextSize = 13,
        Font = TEXT_FONT,
        AutoButtonColor = false,
    }, holder)

    create("UICorner", { CornerRadius = UDim.new(0, 4) }, button)
    local buttonStroke = addStroke(button, DIM_TEXT_COLOR, 0.6)

    button.MouseEnter:Connect(function()
        tween(buttonStroke, FAST_TWEEN, { Color = ACCENT_A, Transparency = 0 })
        tween(button, FAST_TWEEN, { BackgroundColor3 = Color3.fromRGB(28, 28, 30) })
    end)

    button.MouseLeave:Connect(function()
        tween(buttonStroke, FAST_TWEEN, { Color = DIM_TEXT_COLOR, Transparency = 0.6 })
        tween(button, FAST_TWEEN, { BackgroundColor3 = ELEMENT_COLOR })
    end)

    button.MouseButton1Down:Connect(function()
        tween(button, FAST_TWEEN, { BackgroundColor3 = Color3.fromRGB(46, 46, 48) })
    end)

    button.MouseButton1Up:Connect(function()
        tween(button, FAST_TWEEN, { BackgroundColor3 = Color3.fromRGB(28, 28, 30) })
    end)

    button.Activated:Connect(callback)
    table.insert(TemplateHub.searchables, holder)
end

---Start a group of rows that a button shows or hides. The rows that follow belong to the group.
local function beginGroup(parent, text)
    local group = { cards = {}, collapsed = true }
    local holder = createCard("Frame", parent, text .. " Options", 30)

    local button = create("TextButton", {
        Size = UDim2.new(1, -16, 0, 22),
        Position = UDim2.fromOffset(8, 4),
        BackgroundColor3 = ELEMENT_COLOR,
        BorderSizePixel = 0,
        Text = "+  Show options",
        TextColor3 = OFF_TEXT_COLOR,
        TextSize = 12,
        Font = TEXT_FONT,
        AutoButtonColor = false,
    }, holder)

    create("UICorner", { CornerRadius = UDim.new(0, 4) }, button)
```

```
    local buttonStroke = addStroke(button, DIM_TEXT_COLOR, 0.6)

    button.MouseEnter:Connect(function()
        tween(buttonStroke, FAST_TWEEN, { Color = ACCENT_A, Transparency = 0 })
    end)

    button.MouseLeave:Connect(function()
        tween(buttonStroke, FAST_TWEEN, { Color = DIM_TEXT_COLOR, Transparency = 0.6 })
    end)

    button.Activated:Connect(function()
        group.collapsed = not group.collapsed
        button.Text = group.collapsed and "+  Show options" or "-  Hide options"

        for _, card in group.cards do
            card:SetAttribute("GroupHidden", group.collapsed)
            card.Visible = not group.collapsed
        end
    end)

    table.insert(TemplateHub.searchables, holder)
    TemplateHub.currentGroup = group
end

---End the current group of rows.
local function endGroup()
    TemplateHub.currentGroup = nil
end

---Create a scrolling column.
local function createColumn(parent, position)
    local column = create("ScrollingFrame", {
        Size = UDim2.new(0.5, -6, 1, 0),
        Position = position,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        ScrollBarThickness = 3,
        ScrollBarImageColor3 = ACCENT_A,
        ScrollBarImageTransparency = 0.4,
        CanvasSize = UDim2.new(),
        AutomaticCanvasSize = Enum.AutomaticSize.Y,
    }, parent)

    create("UIListLayout", { Padding = UDim.new(0, 4), SortOrder = Enum.SortOrder.LayoutOrder }, column)
    create("UIPadding", { PaddingRight = UDim.new(0, 6) }, column)

    return column
end

---Make a frame draggable by a handle.
local function makeDraggable(handle, target)
    local dragStart, startPosition

    handle.InputBegan:Connect(function(input)
        if not isPress(input) then
            return
        end

        dragStart = input.Position
        startPosition = target.Position
    end)

    connectInput(userInputService.InputChanged, function(input)
        if not dragStart or not isMove(input) then
            return
        end

        local delta = input.Position - dragStart

        target.Position = UDim2.new(
            startPosition.X.Scale,
            startPosition.X.Offset + delta.X,
            startPosition.Y.Scale,
            startPosition.Y.Offset + delta.Y
        )
    end)

    connectInput(userInputService.InputEnded, function(input)
        if isPress(input) then
            dragStart = nil
        end
    end)
end
```

```
---Create a title bar button with hover and press effects.
local function createTitleButton(parent, text, offset, hoverColor)
    local button = create("TextButton", {
        Size = UDim2.fromOffset(24, 22),
        Position = UDim2.new(1, offset, 0.5, -11),
        BackgroundColor3 = ELEMENT_COLOR,
        BackgroundTransparency = 0.4,
        BorderSizePixel = 0,
        Text = text,
        TextColor3 = TEXT_COLOR,
        TextSize = 13,
        Font = BOLD_FONT,
        AutoButtonColor = false,
    }, parent)

    create("UICorner", { CornerRadius = UDim.new(0, 5) }, button)

    button.MouseEnter:Connect(function()
        tween(button, FAST_TWEEN, { BackgroundColor3 = hoverColor, BackgroundTransparency = 0.1 })
    end)

    button.MouseLeave:Connect(function()
        tween(button, FAST_TWEEN, { BackgroundColor3 = ELEMENT_COLOR, BackgroundTransparency = 0.4 })
    end)

    button.MouseButton1Down:Connect(function()
        tween(button, FAST_TWEEN, { BackgroundTransparency = 0.4 })
    end)

    return button
end

---Switch the visible tab.
local function switchTab(name)
    for pageName, page in TemplateHub.pages do
        local selected = pageName == name

        page.Visible = selected
        tween(TemplateHub.tabButtons[pageName], FAST_TWEEN, { TextColor3 = selected and TEXT_COLOR or DIM_TEXT_COLOR })
        if selected then
            page.GroupTransparency = 1
            tween(page, MEDIUM_TWEEN, { GroupTransparency = 0 })
        end
    end

    TemplateHub.selected = name
    tween(TemplateHub.indicator, MEDIUM_TWEEN, { Position = UDim2.new((TemplateHub.tabIndex[name] - 1) / #TAB_NAMES, 6, 0, 3) })
end

---Build the whole interface.
local function initializeScript()
    -- Remove a previous copy of the GUI when the script runs again.
    if typeof(shared) == "table" and shared.TemplateHubGui then
        pcall(function()
            shared.TemplateHubGui:Destroy()
        end)
    end

    -- Prefer the hidden GUI container when the executor provides one.
    local guiParent = localPlayer:WaitForChild("PlayerGui")

    if typeof(gethui) == "function" then
        local success, hiddenParent = pcall(gethui)

        if success and hiddenParent then
            guiParent = hiddenParent
        end
    end

    local screenGui = create("ScreenGui", {
        Name = "TemplateHub",
        ResetOnSpawn = false,
        IgnoreGuiInset = true,
        ZIndexBehavior = Enum.ZIndexBehavior.Sibling,
    }, guiParent)

    if typeof(shared) == "table" then
        shared.TemplateHubGui = screenGui
        shared.TemplateHub = TemplateHub
    end
```

```
    -- Stop every global listener when the GUI is destroyed.
    screenGui.Destroying:Connect(function()
        for _, connection in TemplateHub.connections do
            connection:Disconnect()
        end

        table.clear(TemplateHub.connections)
    end)

    -- Stacked notifications in the bottom right corner.
    local toastHolder = create("Frame", {
        AnchorPoint = Vector2.new(1, 1),
        Position = UDim2.new(1, -16, 1, -16),
        Size = UDim2.fromOffset(230, 320),
        BackgroundTransparency = 1,
        ZIndex = 20,
    }, screenGui)

    create("UIListLayout", {
        Padding = UDim.new(0, 6),
        SortOrder = Enum.SortOrder.LayoutOrder,
        HorizontalAlignment = Enum.HorizontalAlignment.Right,
        VerticalAlignment = Enum.VerticalAlignment.Bottom,
    }, toastHolder)

    ---Show a toast for a toggled feature.
    TemplateHub.notify = function(name, enabled)
        if not toastHolder.Parent then
            return
        end

        local toast = create("CanvasGroup", {
            Size = UDim2.fromOffset(230, 34),
            BackgroundColor3 = Color3.fromRGB(18, 18, 20),
            BorderSizePixel = 0,
            GroupTransparency = 1,
        }, toastHolder)

        create("UICorner", { CornerRadius = UDim.new(0, 7) }, toast)
        addStroke(toast, Color3.new(1, 1, 1), 0.8)
        create("Frame", {
            Size = UDim2.new(0, 3, 1, -12),
            Position = UDim2.fromOffset(6, 6),
            BackgroundColor3 = enabled and Color3.fromRGB(120, 220, 140) or Color3.fromRGB(230, 100, 100),
            BorderSizePixel = 0,
        }, toast)

        create("TextLabel", {
            Size = UDim2.new(1, -80, 1, 0),
            Position = UDim2.fromOffset(18, 0),
            BackgroundTransparency = 1,
            Text = name,
            TextColor3 = TEXT_COLOR,
            TextSize = 13,
            Font = TEXT_FONT,
            TextXAlignment = Enum.TextXAlignment.Left,
            TextTruncate = Enum.TextTruncate.AtEnd,
        }, toast)

        create("TextLabel", {
            Size = UDim2.fromOffset(46, 34),
            Position = UDim2.new(1, -54, 0, 0),
            BackgroundTransparency = 1,
            Text = enabled and "ON" or "OFF",
            TextColor3 = enabled and Color3.fromRGB(120, 220, 140) or Color3.fromRGB(230, 100, 100),
            TextSize = 12,
            Font = BOLD_FONT,
            TextXAlignment = Enum.TextXAlignment.Right,
        }, toast)

        tween(toast, FAST_TWEEN, { GroupTransparency = 0 })

        task.delay(2, function()
            tween(toast, MEDIUM_TWEEN, { GroupTransparency = 1 }).Completed:Wait()
            toast:Destroy()
        end)
    end

    local window = create("Frame", {
        Size = UDim2.fromOffset(WINDOW_WIDTH, WINDOW_HEIGHT),
        Position = UDim2.new(0.5, -WINDOW_WIDTH / 2, 0.5, -WINDOW_HEIGHT / 2),
        BackgroundColor3 = Color3.new(1, 1, 1),
```

```
        BorderSizePixel = 0,
        ClipsDescendants = true,
    }, screenGui)

    create("UICorner", { CornerRadius = UDim.new(0, 8) }, window)

    create("UIGradient", {
        Color = ColorSequence.new(BACKGROUND_TOP, BACKGROUND_BOTTOM),
        Rotation = 90,
    }, window)

    local windowStroke = addStroke(window, Color3.new(1, 1, 1), 0.2, 1.5)
    addGradient(windowStroke, 45)

    local windowScale = create("UIScale", { Scale = 0.85 }, window)
    tween(windowScale, POP_TWEEN, { Scale = 1 })

    -- Top sheen for depth.
    local sheen = create("Frame", {
        Size = UDim2.new(1, 0, 0, 140),
        BackgroundColor3 = Color3.new(1, 1, 1),
        BorderSizePixel = 0,
    }, window)

    create("UIGradient", {
        Transparency = NumberSequence.new(0.965, 1),
        Rotation = 90,
    }, sheen)

    local titleBar = create("Frame", {
        Size = UDim2.new(1, 0, 0, TITLE_HEIGHT),
        BackgroundColor3 = Color3.new(1, 1, 1),
        BorderSizePixel = 0,
    }, window)

    local titleGradient = addGradient(titleBar, 0)
    titleGradient.Transparency = NumberSequence.new(0.8, 0.92)

    local titleLine = create("Frame", {
        Size = UDim2.new(1, 0, 0, 1),
        Position = UDim2.new(0, 0, 1, -1),
        BackgroundColor3 = Color3.new(1, 1, 1),
        BorderSizePixel = 0,
    }, titleBar)

    addGradient(titleLine, 0)

    create("TextLabel", {
        Size = UDim2.new(1, -260, 1, 0),
        Position = UDim2.fromOffset(12, 0),
        BackgroundTransparency = 1,
        RichText = true,
        Text = "<b>" .. TITLE_TEXT .. "</b>",
        TextColor3 = TEXT_COLOR,
        TextSize = 14,
        Font = BOLD_FONT,
        TextXAlignment = Enum.TextXAlignment.Left,
    }, titleBar)

    local searchBox = create("TextBox", {
        Size = UDim2.fromOffset(130, 22),
        Position = UDim2.new(1, -200, 0.5, -11),
        BackgroundColor3 = ELEMENT_COLOR,
        BackgroundTransparency = 0.2,
        BorderSizePixel = 0,
        PlaceholderText = "Search",
        PlaceholderColor3 = DIM_TEXT_COLOR,
        Text = "",
        TextColor3 = TEXT_COLOR,
        TextSize = 12,
        Font = TEXT_FONT,
        ClearTextOnFocus = false,
    }, titleBar)

    create("UICorner", { CornerRadius = UDim.new(0, 5) }, searchBox)
    local searchStroke = addStroke(searchBox, ACCENT_A, 1)

    searchBox.Focused:Connect(function()
        tween(searchStroke, FAST_TWEEN, { Transparency = 0 })
    end)

    searchBox.FocusLost:Connect(function()
        tween(searchStroke, FAST_TWEEN, { Transparency = 1 })
```

```
    end)

    local minimizeButton = createTitleButton(titleBar, "-", -60, ACCENT_B)
    local closeButton = createTitleButton(titleBar, "X", -32, Color3.fromRGB(230, 60, 70))

    makeDraggable(titleBar, window)

    local minimized = false
    local windowWidth = WINDOW_WIDTH
    local windowHeight = WINDOW_HEIGHT

    minimizeButton.Activated:Connect(function()
        minimized = not minimized
        TemplateHub.resizeHandle.Visible = not minimized
        tween(window, MEDIUM_TWEEN, { Size = UDim2.fromOffset(windowWidth, minimized and TITLE_HEIGHT or windowHeight) })
    end)

    closeButton.Activated:Connect(function()
        tween(windowScale, FAST_TWEEN, { Scale = 0 }).Completed:Connect(function()
            screenGui:Destroy()
        end)
    end)

    -- Tab bar with an animated indicator.
    local tabBar = create("Frame", {
        Size = UDim2.new(1, 0, 0, TAB_HEIGHT),
        Position = UDim2.fromOffset(0, TITLE_HEIGHT),
        BackgroundTransparency = 1,
    }, window)

    local indicator = create("Frame", {
        Size = UDim2.new(1 / #TAB_NAMES, -12, 1, -6),
        Position = UDim2.new(0, 6, 0, 3),
        BackgroundColor3 = Color3.new(1, 1, 1),
        BackgroundTransparency = 0.9,
        BorderSizePixel = 0,
    }, tabBar)

    create("UICorner", { CornerRadius = UDim.new(0, 6) }, indicator)
    addStroke(indicator, Color3.new(1, 1, 1), 0.8)

    local underline = create("Frame", {
        AnchorPoint = Vector2.new(0.5, 1),
        Size = UDim2.new(0.5, 0, 0, 2),
        Position = UDim2.fromScale(0.5, 1),
        BackgroundColor3 = Color3.new(1, 1, 1),
        BorderSizePixel = 0,
    }, indicator)

    create("UICorner", { CornerRadius = UDim.new(1, 0) }, underline)
    addGradient(underline, 0)
    TemplateHub.indicator = indicator

    local contentTop = TITLE_HEIGHT + TAB_HEIGHT + 8

    local content = create("Frame", {
        Size = UDim2.new(1, -20, 1, -(contentTop + 10)),
        Position = UDim2.fromOffset(10, contentTop),
        BackgroundTransparency = 1,
    }, window)

    -- Resize grip in the bottom right corner.
    local resizeHandle = create("TextButton", {
        AnchorPoint = Vector2.new(1, 1),
        Size = UDim2.fromOffset(18, 18),
        Position = UDim2.fromScale(1, 1),
        BackgroundTransparency = 1,
        Text = "",
        ZIndex = 10,
    }, window)

    TemplateHub.resizeHandle = resizeHandle

    for gripIndex = 1, 3 do
        create("Frame", {
            AnchorPoint = Vector2.new(1, 1),
            Size = UDim2.fromOffset(gripIndex * 4, 1),
            Position = UDim2.new(1, -3, 1, -3 - (3 - gripIndex) * 4 - 2),
            BackgroundColor3 = DIM_TEXT_COLOR,
            BorderSizePixel = 0,
        }, resizeHandle)
    end
```

```
    local resizing = false
    local resizeStart, startWidth, startHeight

    resizeHandle.MouseEnter:Connect(function()
        for _, grip in resizeHandle:GetChildren() do
            tween(grip, FAST_TWEEN, { BackgroundColor3 = ACCENT_A })
        end
    end)

    resizeHandle.MouseLeave:Connect(function()
        if resizing then
            return
        end

        for _, grip in resizeHandle:GetChildren() do
            tween(grip, FAST_TWEEN, { BackgroundColor3 = DIM_TEXT_COLOR })
        end
    end)

    resizeHandle.InputBegan:Connect(function(input)
        if not isPress(input) then
            return
        end

        resizing = true
        resizeStart = input.Position
        startWidth, startHeight = windowWidth, windowHeight
    end)

    connectInput(userInputService.InputChanged, function(input)
        if not resizing or not isMove(input) then
            return
        end

        local delta = input.Position - resizeStart
        windowWidth = math.clamp(startWidth + delta.X, MIN_WIDTH, MAX_WIDTH)
        windowHeight = math.clamp(startHeight + delta.Y, MIN_HEIGHT, MAX_HEIGHT)
        window.Size = UDim2.fromOffset(windowWidth, windowHeight)
    end)

    connectInput(userInputService.InputEnded, function(input)
        if not resizing or not isPress(input) then
            return
        end

        resizing = false

        for _, grip in resizeHandle:GetChildren() do
            tween(grip, FAST_TWEEN, { BackgroundColor3 = DIM_TEXT_COLOR })
        end
    end)

    for index, name in TAB_NAMES do
        local button = create("TextButton", {
            Size = UDim2.fromScale(1 / #TAB_NAMES, 1),
            Position = UDim2.fromScale((index - 1) / #TAB_NAMES, 0),
            BackgroundTransparency = 1,
            Text = name,
            TextColor3 = DIM_TEXT_COLOR,
            TextSize = 13,
            Font = BOLD_FONT,
            AutoButtonColor = false,
        }, tabBar)

        local page = create("CanvasGroup", {
            Size = UDim2.fromScale(1, 1),
            BackgroundTransparency = 1,
            Visible = false,
        }, content)

        button.MouseEnter:Connect(function()
            if TemplateHub.selected ~= name then
                tween(button, FAST_TWEEN, { TextColor3 = OFF_TEXT_COLOR })
            end
        end)

        button.MouseLeave:Connect(function()
            if TemplateHub.selected ~= name then
                tween(button, FAST_TWEEN, { TextColor3 = DIM_TEXT_COLOR })
            end
        end)

        button.Activated:Connect(function()
```

```
            switchTab(name)
        end)

        TemplateHub.pages[name] = page
        TemplateHub.tabButtons[name] = button
        TemplateHub.tabIndex[name] = index
    end

    -- Tab1.
    local tab1Left = createColumn(TemplateHub.pages.Tab1, UDim2.fromOffset(0, 0))
    local tab1Right = createColumn(TemplateHub.pages.Tab1, UDim2.new(0.5, 6, 0, 0))

    addSection(tab1Left, "Section1")
    addToggle(tab1Left, "Template1", nil, true)
    addToggle(tab1Left, "Template2", nil, true)
    addToggle(tab1Left, "Template3", nil, true)
    addCycle(tab1Left, "Template4", { "Option1", "Option2", "Option3", "Option4" })
    addToggle(tab1Left, "Template5", nil, true)

    addSection(tab1Left, "Section2")
    addSlider(tab1Left, "Template6", 0, 10, 1, 0)
    addSlider(tab1Left, "Template7", 0, 300, 120, 0)
    addSlider(tab1Left, "Template8", 0, 300, 80, 0)
    addSlider(tab1Left, "Template9", 0, 300, 0, 0)

    addSection(tab1Right, "Section3")
    addToggle(tab1Right, "Template10", nil, true)
    addToggle(tab1Right, "Template11", nil, true)
    addToggle(tab1Right, "Template12", nil, true)

    addSection(tab1Right, "Section4")
    addToggle(tab1Right, "Template13", nil, true)
    addToggle(tab1Right, "Template14", nil, true)

    -- Tab2.
    local tab2Left = createColumn(TemplateHub.pages.Tab2, UDim2.fromOffset(0, 0))
    local tab2Right = createColumn(TemplateHub.pages.Tab2, UDim2.new(0.5, 6, 0, 0))

    addSection(tab2Left, "Section5")
    addToggle(tab2Left, "Template15", nil, true)
    beginGroup(tab2Left, "Group1")
    addSlider(tab2Left, "Template16", 5, 45, 22, 0)
    addSlider(tab2Left, "Template17", 0.05, 1, 0.3, 2)
    addToggle(tab2Left, "Template18", nil, true)
    addToggle(tab2Left, "Template19", nil, true)
    addSlider(tab2Left, "Template20", 3, 30, 18, 0)
    addSlider(tab2Left, "Template21", 0, 100, 100, 0)
    addToggle(tab2Left, "Template22", nil, true)
    addToggle(tab2Left, "Template23", nil, true)
    addSlider(tab2Left, "Template24", 0, 1, 0, 2)
    addSlider(tab2Left, "Template25", 0, 1, 0.1, 2)
    endGroup()

    addSection(tab2Left, "Section6")
    addToggle(tab2Left, "Template26", nil, true)
    beginGroup(tab2Left, "Group2")
    addToggle(tab2Left, "Template27", nil, true)
    addToggle(tab2Left, "Template28", nil, true)
    addSlider(tab2Left, "Template29", 1, 28, 28, 0)
    addSlider(tab2Left, "Template30", 10, 270, 180, 0)
    addSlider(tab2Left, "Template31", 0, 2, 0, 2)
    addSlider(tab2Left, "Template32", 0, 2, 0.15, 2)
    addToggle(tab2Left, "Template33", nil, true)
    addToggle(tab2Left, "Template34", nil, true)
    addSlider(tab2Left, "Template35", 1, 28, 12, 0)
    addToggle(tab2Left, "Template36", nil, true)
    endGroup()

    addSection(tab2Left, "Section7")
    addToggle(tab2Left, "Template37", nil, true)
    beginGroup(tab2Left, "Group3")
    addToggle(tab2Left, "Template38", nil, true)
    addToggle(tab2Left, "Template39", nil, true)
    addSlider(tab2Left, "Template40", 1, 28, 28, 0)
    addSlider(tab2Left, "Template41", 10, 270, 180, 0)
    addToggle(tab2Left, "Template42", nil, true)
    addToggle(tab2Left, "Template43", nil, true)
    addToggle(tab2Left, "Template44", nil, true)
    addToggle(tab2Left, "Template45", nil, true)
    endGroup()

    addSection(tab2Right, "Section8")
    addToggle(tab2Right, "Template46", nil, true)
```

```
    addToggle(tab2Right, "Template47", nil, true)
    addSlider(tab2Right, "Template48", 0, 25, 16, 0)

    addSection(tab2Right, "Section9")
    addToggle(tab2Right, "Template49", nil, true)
    addSlider(tab2Right, "Template50", 0, 24, 12, 1)

    addSection(tab2Right, "Section10")
    addToggle(tab2Right, "Template51", nil, true)

    -- Tab3.
    local tab3Left = createColumn(TemplateHub.pages.Tab3, UDim2.fromOffset(0, 0))
    local tab3Right = createColumn(TemplateHub.pages.Tab3, UDim2.new(0.5, 6, 0, 0))

    addSection(tab3Left, "Section11")
    addToggle(tab3Left, "Template52", nil, true)
    addToggle(tab3Left, "Template53", nil, true)
    addToggle(tab3Left, "Template54", nil, false)
    addSlider(tab3Left, "Template55", 50, 2000, 500, 0)

    addSection(tab3Right, "Section12")
    addCycle(tab3Right, "Template56", { "Option1", "Option2", "Option3", "Option4", "Option5" })
    addSlider(tab3Right, "Template57", 0, 1, 0.3, 2)

    -- Config tab.
    local configLeft = createColumn(TemplateHub.pages.Config, UDim2.fromOffset(0, 0))
    local configRight = createColumn(TemplateHub.pages.Config, UDim2.new(0.5, 6, 0, 0))

    local statusLabel = create("TextLabel", {
        Size = UDim2.new(1, -8, 0, 18),
        Position = UDim2.new(0, 4, 1, -20),
        BackgroundTransparency = 1,
        Text = "",
        TextColor3 = DIM_TEXT_COLOR,
        TextSize = 12,
        Font = TEXT_FONT,
        TextXAlignment = Enum.TextXAlignment.Left,
        ZIndex = 5,
    }, TemplateHub.pages.Config)

    ---Show a short status message.
    local function setStatus(message)
        statusLabel.Text = message
        statusLabel.TextTransparency = 0

        task.delay(3, function()
            if statusLabel.Text == message then
                tween(statusLabel, MEDIUM_TWEEN, { TextTransparency = 1 })
            end
        end)
    end

    ---Save every setting to a file.
    local function saveSettings()
        if not writefile then
            return setStatus("Saving is not supported here.")
        end

        local ok, result = pcall(function()
            if makefolder and isfolder and not isfolder(CONFIG_FOLDER) then
                makefolder(CONFIG_FOLDER)
            end

            writefile(CONFIG_FILE, httpService:JSONEncode(TemplateHub.values))
        end)

        setStatus(ok and "Settings saved." or ("Save failed: " .. tostring(result)))
    end

    ---Load every setting from the file.
    local function loadSettings()
        if not (readfile and isfile and isfile(CONFIG_FILE)) then
            return setStatus("No saved settings found.")
        end

        local ok, data = pcall(function()
            return httpService:JSONDecode(readfile(CONFIG_FILE))
        end)

        if not ok or typeof(data) ~= "table" then
            return setStatus("Settings file is corrupted.")
        end
```

```
        for name, value in data do
            local setter = TemplateHub.setters[name]

            if setter then
                pcall(setter, value)
            end
        end

        setStatus("Settings loaded.")
    end

    ---Reset every setting to its default.
    local function resetSettings()
        for name, value in TemplateHub.defaults do
            TemplateHub.setters[name](value)
        end

        setStatus("Settings reset.")
    end

    addSection(configLeft, "Settings")
    addButton(configLeft, "Save Settings", saveSettings)
    addButton(configLeft, "Load Settings", loadSettings)
    addButton(configLeft, "Reset Settings", resetSettings)
    addButton(configLeft, "Copy Settings", function()
        if not setclipboard then
            return setStatus("Clipboard is not supported here.")
        end

        setclipboard(httpService:JSONEncode(TemplateHub.values))
        setStatus("Settings copied.")
    end)
    addToggle(configLeft, "Auto Save", nil, false)

    addSection(configRight, "Interface")
    addCycle(configRight, "Menu Keybind", { "RightShift", "Insert", "RightControl", "Delete", "End" })
    addButton(configRight, "Reset Window", function()
        windowWidth, windowHeight = WINDOW_WIDTH, WINDOW_HEIGHT
        minimized = false
        TemplateHub.resizeHandle.Visible = true
        tween(window, MEDIUM_TWEEN, {
            Size = UDim2.fromOffset(windowWidth, windowHeight),
            Position = UDim2.new(0.5, -windowWidth / 2, 0.5, -windowHeight / 2),
        })
    end)
    addButton(configRight, "Unload GUI", function()
        tween(windowScale, FAST_TWEEN, { Scale = 0 }).Completed:Connect(function()
            screenGui:Destroy()
        end)
    end)

    -- Apply the menu keybind and autosave when enabled.
    task.spawn(function()
        while screenGui.Parent do
            local keyName = TemplateHub.values["Menu Keybind"]
            TemplateHub.menuKey = Enum.KeyCode.RightShift

            for _, item in Enum.KeyCode:GetEnumItems() do
                if item.Name == keyName then
                    TemplateHub.menuKey = item
                    break
                end
            end

            if TemplateHub.values["Auto Save"] and writefile then
                pcall(function()
                    if makefolder and isfolder and not isfolder(CONFIG_FOLDER) then
                        makefolder(CONFIG_FOLDER)
                    end

                    writefile(CONFIG_FILE, httpService:JSONEncode(TemplateHub.values))
                end)
            end

            task.wait(5)
        end
    end)

    searchBox:GetPropertyChangedSignal("Text"):Connect(function()
        local query = string.lower(searchBox.Text)

        for _, element in TemplateHub.searchables do
```

```
            element.Visible = (query == "" or string.find(string.lower(element.Name), query, 1, true) ~= nil) and not element:GetAttribute("GroupHidden")
        end

        for _, section in TemplateHub.sections do
            section.Visible = query == ""
        end

        for _, box in TemplateHub.boxes do
            local anyVisible = query == ""

            for _, child in box:GetChildren() do
                if child:IsA("GuiObject") and child.Visible and not table.find(TemplateHub.sections, child) then
                    anyVisible = true
                    break
                end
            end

            box.Visible = anyVisible
        end
    end)

    -- The menu keybind hides and shows the menu.
    connectInput(userInputService.InputBegan, function(input, gameProcessed)
        if input.UserInputType ~= Enum.UserInputType.Keyboard then
            return
        end

        -- A keybind button is waiting for the next key.
        if TemplateHub.listening then
            return TemplateHub.listening(input.KeyCode)
        end

        if gameProcessed then
            return
        end

        if input.KeyCode == TemplateHub.menuKey then
            screenGui.Enabled = not screenGui.Enabled
            return
        end

        -- Toggle any feature bound to this key.
        for featureName, keyCode in TemplateHub.keybinds do
            if keyCode == input.KeyCode then
                TemplateHub.setters[featureName](not TemplateHub.values[featureName])
                TemplateHub.notify(featureName, TemplateHub.values[featureName])
            end
        end
    end)

    -- Keybind Hints shows or hides the keybind labels.
    task.spawn(function()
        while screenGui.Parent do
            local visible = TemplateHub.values["Keybind Hints"] ~= false

            for _, keybindButton in TemplateHub.keyLabels do
                keybindButton.Visible = visible
            end

            task.wait(0.5)
        end
    end)

    switchTab("Tab1")
end

---Called when initialization errors.
local function onInitializeError(error)
    warn("Failed to initialize.")
    warn(error)
    warn(debug.traceback())
end

xpcall(initializeScript, onInitializeError)
```